OBJECT ORIENTED ANALYSIS AND DESIGN (CT 651)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 3: Object Oriented Design

Syllabus Hours: 12 hrs | Marks Weight: 21 marks

CHAPTER 3 SYLLABUS TOPICS

3.1 Analysis to Design
3.2 Describing and Elaborating Use Cases
3.3 Collaboration Diagram
3.4 Objects and Patterns (GRASP Patterns)
3.5 Determining Visibility
3.6 Class Diagram

SECTION 1: Analysis to Design — Transition from OOA to OOD

Exam Ref: 2082 Kartik Q5 [5], 2081 Chaitra Q6 [5], 2081 Ashwin Q5 [5], 2080 Chaitra Q5, 2079 Ashwin Q5 [5+5], 2078 Baishakh Q5 [4], 2077 Chaitra Q5, 2075 Baisakh Q3

Q1. Explain the process of transition from Analysis to Design in Object Oriented Development. [5] (2082 Kartik)

Answer:

Overview — From Analysis to Design

The transition from OOA (Object Oriented Analysis) to OOD (Object Oriented Design) is the central activity in OO development. OOA asks WHAT the system must do, while OOD answers HOW the system will do it. This transition involves expanding the conceptual model into a full design model suitable for implementation.

Key Transitions from OOA to OOD:

OOA Artifact	OOD Artifact	Transition Activity
Domain Model (Conceptual Classes)	Design Class Diagram	Add methods, types, visibility, navigability to conceptual classes.
Use Cases (text)	Interaction Diagrams (Sequence/Collaboration)	Elaborate use case steps into object method calls and message flows.
System Sequence Diagram (SSD)	Detailed Sequence / Collaboration Diagrams	Expand system input events into actual object interactions.
System Operations (from SSD)	Controller Methods + GRASP-assigned Responsibilities	Apply GRASP patterns to assign each responsibility to the correct class.
Conceptual Associations	Design Associations with navigability	Decide which direction associations need to navigate.
Actors	UI / Boundary Objects	Actor interactions become boundary class operations.

Steps in the Analysis-to-Design Transition:

Step 1 — Take SSD system events (e.g., enterItem(itemID, qty)) and assign to a Controller class using GRASP.

Step 2 — Create Interaction Diagrams (collaboration or sequence) showing which objects collaborate and how messages flow.

Step 3 — Identify responsibilities (know/do) for each object participating in interactions.

Step 4 — Update the Design Class Diagram: add methods discovered from interactions, add attributes with types, add visibility.

Step 5 — Apply GRASP patterns (Information Expert, Creator, Controller, Low Coupling, High Cohesion) to guide all design decisions.

Step 6 — Define operation contracts to verify pre/post conditions are met by the design.

OOA vs. OOD — Key Differences:

Aspect	OOA	OOD
Focus	WHAT the system must do	HOW the system will do it
Classes	Conceptual classes (no methods)	Design classes (with methods, types, visibility)
Diagrams	Domain Model, Use Case Diagram, SSD	Collaboration/Sequence Diagrams, Design Class Diagram
Relationships	Conceptual associations	Navigability, multiplicity, visibility specified
Patterns	Not applied yet	GRASP, GoF patterns applied
Language-specific	Language independent	May be language-specific (Java/C++ constructs)

Q2. Describe in brief the 'Analysis to Design' process using GRASP. How does the interaction diagram assist in the design phase? [5+5] (2079 Ashwin Back)

Answer:

Analysis to Design Using GRASP

GRASP (General Responsibility Assignment Software Patterns) is the fundamental framework used during the analysis-to-design transition. It provides proven principles for deciding which class should be responsible for a given operation or knowledge.

Process:

1. Identify system operations from SSD (e.g., makeNewSale(), enterItem(), makePayment()).
2. For each operation, use GRASP Information Expert to find which class has the information needed to fulfill the responsibility.
3. Use GRASP Creator to decide which class should create instances of other classes.
4. Use GRASP Controller to assign the system operation handler.
5. Draw an Interaction Diagram showing how objects collaborate to fulfill each operation.
6. From Interaction Diagrams, extract method signatures and add to Design Class Diagram.

How Interaction Diagrams Assist in Design:

Role of Interaction Diagram	Explanation
Assign responsibilities to objects	Shows which object sends which message — revealing method assignments.
Discover methods	Every message sent to an object becomes a method on that object.
Show object collaboration	Reveals which objects need references to other objects (navigability in class diagram).
Verify design correctness	Can be traced back to SSD to verify all system events are handled.
Low Coupling check	Too many arrows from one object = high coupling — needs redesign.
Capture design decisions	Documents WHY a particular object was chosen for a responsibility.

SECTION 2: GRASP Patterns — Objects and Patterns

Exam Ref: 2082 Kartik Q6 [10], 2081 Chaitra Q7 [10], 2081 Ashwin Q6 [5+5], 2080 Chaitra Q6 [10], 2080 Ashwin Q6 [10], 2079 Chaitra Q6 [10], 2079 Ashwin Q6 [5+5], 2078 Chaitra Q6 [10], 2078 Baishakh Q6 [10], 2077 Chaitra Q6 [10], 2076, 2075, 2074, 2073, 2072

Q3. What are GRASP patterns? Explain each GRASP pattern with suitable example. [10] (2082 Kartik, 2081 Chaitra, 2080 Chaitra, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra — MOST REPEATED QUESTION)

Answer:

GRASP — General Responsibility Assignment Software Patterns

GRASP is a set of 9 fundamental design principles (patterns) that guide the assignment of responsibilities (methods, behaviors) to classes in Object Oriented Design. They were introduced by Craig Larman. The core question GRASP answers is: 'Which class should be responsible for this operation or data?'

GRASP Pattern 1 — Information Expert

Principle: Assign responsibility to the class that has the most information needed to fulfill it — the 'Information Expert'.

Aspect	Detail
Problem	What is the most basic principle by which to assign responsibilities to objects?
Solution	Assign a responsibility to the class that has the information (data) necessary to fulfill it.
Example	Who calculates the total of a Sale? → The Sale class, because it has the SaleLineItems and their prices. So getTotal() goes to Sale.
Benefit	Promotes encapsulation — data and the behavior that uses it are in the same class.
Anti-pattern	Do NOT assign a method to a class that doesn't have the relevant data.

GRASP Pattern 2 — Creator

Principle: Assign class B the responsibility to CREATE an instance of class A if B 'Contains', 'Aggregates', 'Records', 'Closely uses', or 'Has the initializing data' for A.

Aspect	Detail
Problem	Who should be responsible for creating a new instance of a class?
Solution	Assign creation to class B if B contains/aggregates A, OR B records A, OR B closely uses A, OR B has initializing data for A. (Mnemonic: CAREH)
Example	Who creates SaleLineItem? → Sale, because Sale CONTAINS SaleLineItems (aggregation). So Sale.addLineItem() creates SaleLineItem instances.
Example 2	Who creates Payment? → Sale, because Sale RECORDS Payment and has the total to initialize it.
Benefit	Keeps creation responsibility close to the object that needs the new instance.

GRASP Pattern 3 — Controller

Principle: Assign the responsibility for receiving or handling a system event message to a class representing one of:

(a) the overall system/root object (Facade Controller), or

(b) a use-case session handler (Use Case Controller).

Aspect	Detail
Problem	What first object beyond the UI layer receives and coordinates a system operation?
Solution	Assign to a Controller class — either a Facade (represents the whole system, e.g., Register, POSSystem) or a Use Case Session Controller (e.g., ProcessSaleHandler).
Example	makeNewSale(), enterItem(), makePayment() → all handled by Register (Facade Controller), NOT by the Sale object or UI.
Benefit	Separates UI logic from application/domain logic. Makes the system testable without UI.

Bloated Controller	If Controller does too much logic itself → split into Use Case Controllers or delegate to domain objects.

GRASP Pattern 4 — Low Coupling

Principle: Assign responsibilities so that coupling (dependency between classes) remains LOW. Classes should not depend on too many other classes.

Aspect	Detail
Problem	How to reduce the impact of change? How to support low dependency and increased reuse?
Solution	Assign responsibilities such that unnecessary coupling is avoided. Prefer to give responsibility to a class that already has coupling to the relevant classes.
Example	Payment should be created by Sale (not by Register), because Sale already knows about the total. Making Register create Payment would add unnecessary coupling from Register to Payment.
Benefit	Changes in one class don't cascade to many others. Easier to reuse classes in isolation.
Tradeoff	Don't sacrifice clarity or correctness for low coupling.

GRASP Pattern 5 — High Cohesion

Principle: Assign responsibilities so that cohesion (focus of a class) remains HIGH. A class should do one thing well — not too many unrelated things.

Aspect	Detail
Problem	How to keep objects focused, understandable, and manageable, and as a side effect support Low Coupling?
Solution	A class should have a small set of strongly related responsibilities. A class doing too many things should be split.
Example	Register should NOT handle sale processing AND payment AND printing AND inventory. Instead, separate responsibilities: Sale (handles sale data), PaymentProcessor (handles payment), Printer (handles printing).
Benefit	Classes are easier to understand, maintain, and reuse.
Low Cohesion Signs	Class has too many methods doing unrelated things. Method names don't match class purpose.

GRASP Pattern 6 — Polymorphism

Principle: When behavior varies by type, assign responsibilities to the types involved using polymorphism, rather than using explicit if-else or switch statements.

Aspect	Detail
Problem	How to handle alternatives based on type? How to avoid if-then-else testing of type?
Solution	Use polymorphism (interfaces, abstract classes, method overriding) to vary behavior by type.
Example	Instead of: if (payment instanceof Cash) {...} else if (payment instanceof Card) {...}, define Payment as abstract with pay() method, and CashPayment/CardPayment override pay().
Benefit	Easy to extend with new types without changing existing code (Open/Closed Principle).

GRASP Pattern 7 — Pure Fabrication

Principle: Assign a highly cohesive set of responsibilities to an artificial/convenience class that does NOT represent a real-world concept — a 'made-up' service class — when a domain class would be forced to have low cohesion or high coupling.

Aspect	Detail
Problem	What object should have a responsibility when you don't want to violate Information Expert and High Cohesion, but domain classes are unsuitable?

Solution	Create a new class (Pure Fabrication / Service class) that doesn't represent any real-world thing, but has high cohesion for the service it provides.
Example	Who should save a Sale to a database? Not Sale itself (it would need DB knowledge — low cohesion). Create a PersistenceService or SaleRepository class.
Benefit	Highly reusable, cohesive service classes that support Low Coupling.

GRASP Pattern 8 — Indirection

Principle: Assign responsibility to an intermediate object to mediate between two components or services, so they are not directly coupled.

Aspect	Detail
Problem	How to avoid direct coupling between two classes? Where to assign a responsibility to avoid direct coupling?
Solution	Create an intermediate/mediator object between the two classes.
Example	Instead of Sale directly calling TaxSystem.getTax(), create a TaxCalculator (indirection object) that mediates. Sale only knows TaxCalculator, not TaxSystem.
Benefit	Components can vary independently. Supports pluggable/replaceable implementations.
Relation	Most GoF patterns (Adapter, Facade, Proxy) use Indirection.

GRASP Pattern 9 — Protected Variations

Principle: Identify points of predicted variation or instability; assign responsibilities to create a stable interface around them.

Aspect	Detail
Problem	How to design objects and systems so that variations/instabilities in these elements do NOT have an undesirable impact on other elements?
Solution	Identify points of variation (requirements that might change). Protect other classes from these variations by wrapping them behind a stable interface/abstract class.
Example	Payment method may change (Cash, Credit, Crypto). Define abstract Payment interface with pay() method. All payment variations are 'protected' behind this interface.
Benefit	System is open for extension (new types) but closed for modification (existing code unchanged).
Relation	This is the foundation of most OO design principles: encapsulation, polymorphism, Open/Closed.

GRASP Quick Summary Table:

Pattern	Core Question	Key Benefit
1. Information Expert	Who has the data needed?	Encapsulation — data and behavior together
2. Creator	Who should create object X?	Maintains natural object relationships
3. Controller	Who handles system events?	Separates UI from domain logic
4. Low Coupling	How to minimize dependencies?	Easier change, higher reuse
5. High Cohesion	How to keep classes focused?	Easier to understand, maintain, test
6. Polymorphism	How to vary behavior by type?	Extensible without modifying existing code
7. Pure Fabrication	Where when domain class is unsuitable?	Reusable, cohesive service classes
8. Indirection	How to avoid direct coupling?	Pluggable, replaceable components
9. Protected Variations	How to protect against change?	Stable design open to extension

Q4. Apply GRASP patterns to design the Process Sale use case. Show which class gets each responsibility and why. [10] (2080 Ashwin, 2081 Ashwin)

Answer:

Applying GRASP to Process Sale Use Case

The POS Process Sale use case has 4 system operations from the SSD: makeNewSale(), enterItem(itemID, qty), endSale(), makePayment(cashTendered).

System Operation	GRASP Pattern Applied	Responsible Class	Justification
makeNewSale()	Controller	Register	Register is the Facade Controller for POS system. It represents the physical cash register.
makeNewSale()	Creator	Register creates Sale	Register RECORDS Sales and is the natural container. Creator: Register aggregates/records Sale.
enterItem(itemID, qty)	Controller	Register	Register handles all system operations as Facade Controller.
enterItem() — find ProductDesc	Information Expert	ProductCatalog	ProductCatalog has info about all products. Expert: it contains ProductDescriptions.
enterItem() — create SaleLineItem	Creator	Sale creates SaleLineItem	Sale CONTAINS SaleLineItems. Creator: Sale aggregates SaleLineItems.
getTotal()	Information Expert	Sale	Sale CONTAINS SaleLineItems and their quantities. Expert: it has all the data needed.
getSubtotal()	Information Expert	SaleLineItem	SaleLineItem HAS quantity and its ProductDescription with price.
makePayment()	Controller	Register	Facade Controller receives all system events.
makePayment() — create Payment	Creator	Sale creates Payment	Sale RECORDS Payment and has the total to initialize it. Creator pattern.

Resulting Design Class Diagram — Methods Discovered:

Class	Methods (discovered from GRASP assignments)
Register	makeNewSale(), enterItem(itemID, qty), endSale(), makePayment(cashTendered)
Sale	makeLineItem(spec, qty), getTotal(), becomeComplete(), makePayment(cashTendered)
SaleLineItem	getSubtotal()
ProductCatalog	getSpecification(itemID): ProductDescription
ProductDescription	getPrice()
Payment	Payment(cashTendered: Money)

SECTION 3: Collaboration Diagram (Communication Diagram)

Exam Ref: 2082 Q7 [5+5], 2081 Chaitra Q8 [5+5], 2081 Ashwin Q7 [5+5], 2080 Chaitra Q7 [10], 2079 Ashwin Q7 [5+5], 2079 Chaitra Q7, 2078 Chaitra Q7 [10], 2078 Baishakh Q7 [5+5], 2077 Chaitra Q7 [10]

Q5. What is a Collaboration Diagram? What are the differences between Collaboration Diagram and Sequence Diagram? Draw a Collaboration Diagram for enterItem() system operation of POS system. [5+5] (2082 Kartik)

Answer:

Collaboration Diagram — Definition

A Collaboration Diagram (also called Communication Diagram in UML 2.x) is an interaction diagram that shows how objects collaborate by sending messages to each other. It emphasizes the structural organization (links between objects) rather than the time ordering of messages. Each message is numbered to show the sequence of calls.

Key Elements of a Collaboration Diagram:

- Objects: Shown as rectangles with format: objectName:ClassName (e.g., r:Register, s:Sale).
- Links: Lines between objects showing that one object can send messages to another.
- Messages: Labeled on links with sequence numbers (e.g., 1: makeNewSale(), 2: makeLineItem()).
- Sequence Numbers: Show the order of messages. Nested calls use dotted notation: 1.1, 1.2, 1.1.1, etc.
- Creation: New object creation shown by <<create>> message.
- Self-calls: An object sending a message to itself.

Collaboration Diagram vs. Sequence Diagram — Comparison:

Aspect	Collaboration Diagram	Sequence Diagram
Also called	Communication Diagram (UML 2.x)	Interaction Diagram
Primary focus	Structural — shows object links and connections	Temporal — shows time ordering of messages
Time ordering	Shown via sequence numbers (1, 1.1, 1.2, 2...)	Shown via vertical position on diagram
Layout	Objects spread out as nodes; links as edges	Objects as columns; time flows downward
Complexity	Harder to follow when many nested calls	Easier to read for time-ordered flows
Space efficiency	More compact — fits more objects	Can get very long vertically
Concurrent flows	Hard to show	Easy to show with parallel lifelines
Best for	Showing which objects communicate with which	Showing detailed time-ordered message flow
UML notation	Boxes connected by lines with numbered messages	Lifelines + horizontal arrows with labels
Semantic equivalence	Both diagrams express the SAME information — just different layouts	Same as collaboration diagram

Collaboration Diagram — enterItem(itemID, qty) System Operation:

Participants: :Register (Controller), :Sale (current sale), :ProductCatalog, :ProductDescription, :SaleLineItem

Step	Sender	Message	Receiver	Notes
1	:Register	enterItem(itemID, qty)	:Sale	Register delegates to Sale (Controller delegates to domain)
1.1	:Sale	spec = getSpecification(itemID)	:ProductCatalog	Sale asks catalog for product description — Information Expert
1.2	:Sale	makeLineItem(spec, qty)	:Sale (self)	Sale creates a new SaleLineItem — Creator pattern

1.2.1	:Sale	<<create>>(spec, qty)	:SaleLineItem	New SaleLineItem object instantiated
1.2.2	:Sale	add(sli)	:Sale.lineItems	SaleLineItem added to Sale's list
1.3	:Register	return description, total	UI/Cashier	Register returns item info to the UI layer

Sequence Numbers Explained: 1 = top level. 1.1 = first sub-call within message 1. 1.2 = second sub-call within message 1. 1.2.1 = first sub-call within message 1.2. Numbers create a tree structure showing nested call depth.

Q6. Draw a Collaboration Diagram for makePayment() system operation of a POS System. Explain the sequence numbering scheme. [5+5] (2081 Chaitra, 2078 Chaitra)

Answer:

Collaboration Diagram — makePayment(cashTendered) System Operation:

Step	Sender	Message	Receiver	GRASP Pattern
1	:Register	makePayment(cashTendered)	:Sale	Controller delegates to domain
1.1	:Sale	<<create>>(cashTendered)	:Payment	Creator — Sale records Payment
1.2	:Sale	becomeComplete()	:Sale (self)	Sale marks itself as completed
1.3	:Sale	getTotal()	:Sale (self)	Information Expert — Sale calculates its total
1.3.1	:Sale	getSubtotal()	:SaleLineItem	Expert — SaleLineItem knows subtotal
1.3.1.1	:SaleLineItem	getPrice()	:ProductDescription	Expert — ProductDesc knows price
2	:Register	printReceipt(sale)	:Printer	Low coupling — Printer handles printing
2.1	:Printer	getLineItems()	:Sale	Printer requests data from Sale

Sequence Numbering Scheme in Collaboration Diagrams:

Notation	Meaning	Example
1	Top-level message (first system operation call)	1: makePayment(cashTendered)
1.1	First sub-call triggered by message 1	1.1: <<create>>(cashTendered)
1.2	Second sub-call triggered by message 1	1.2: becomeComplete()
1.3.1	First sub-call within sub-call 1.3	1.3.1: getSubtotal()
1.3.1.1	Sub-call within 1.3.1 (third nesting level)	1.3.1.1: getPrice()
2	Second top-level message (separate system event)	2: printReceipt(sale)
*	Iteration (loop) — message sent multiple times	1.3.1*: getSubtotal() [for each item]
[condition]	Guard condition	[cashTendered >= total] makeChange()

Q7. Draw Collaboration Diagram for makeNewSale() operation of the POS system. Also draw a sequence diagram for the same operation and compare both. [5+5] (2081 Ashwin Back)

Answer:

Collaboration Diagram — makeNewSale() Operation:

Step	Sender	Message	Receiver	Notes
1	:Register	makeNewSale()	:Register (handler)	Controller — Register handles system event

1.1	:Register	<<create>>( )	:Sale	Creator — Register creates new Sale object
1.2	:Register	setCurrentSale(sale)	:Register (self)	Register stores reference to current Sale
1.3	:Register	return 'newSaleStarted'	UI/Cashier	Response sent back to presentation layer

Sequence Diagram — makeNewSale() (described textually):

- Lifelines: :Cashier (actor), :Register, :Sale
- Message 1: Cashier → :Register: makeNewSale() [solid arrow]
- Message 1.1: :Register → :Sale: <<create>>() [solid arrow, creates new Sale lifeline]
- Message 1.2: :Register → :Register: currentSale = sale [self-call]
- Return: :Register → :Cashier: 'OK, new sale started' [dashed arrow]

Key Comparison — Collaboration vs. Sequence for makeNewSale():

The Collaboration Diagram shows Register linked to Sale with a <<create>> message labeled 1.1. The Sequence Diagram shows the same information but as a vertical timeline — Sale's lifeline appears after the <<create>> message arrow. Both convey the same semantics: Register creates Sale. Collaboration is more compact; Sequence makes timing clearer.

SECTION 4: Describing and Elaborating Use Cases

Exam Ref: 2082 Q2 [8], 2081 Chaitra Q5 [4+4], 2081 Ashwin Q4, 2080 Ashwin Q4, 2079 Chaitra Q5, 2078 Baishakh Q5, 2077 Chaitra Q5 [5+5]

Q8. What are the different sections of a fully-dressed Use Case? Elaborate the 'Process Sale' use case in fully-dressed format. [8] (2082 Kartik)

Answer:

Sections of a Fully-Dressed Use Case

A Fully-Dressed Use Case is a detailed, structured description of a use case that captures all important information about an interaction. It goes far beyond a simple one-line description.

Section	Description
Use Case Name	Verb phrase describing what actor accomplishes (e.g., Process Sale).
Scope	The system being designed (e.g., POS Application).
Level	User-goal level, Sub-function level, or Summary level.
Primary Actor	The stakeholder who initiates the use case and has the goal (e.g., Cashier).
Stakeholders & Interests	All parties who have a stake in the use case and what they want.
Preconditions	What must be true before the use case starts (not verified during execution).
Success Guarantee (Post-conds)	What must be true when the use case completes successfully.
Main Success Scenario	The 'happy path' — numbered steps for the typical successful interaction.
Extensions (Alternate Flows)	Other scenarios — error cases, alternative paths, exceptional cases.
Special Requirements	Non-functional requirements specific to this use case.
Technology & Data Variations	Variations in I/O or technology (e.g., payment by cash OR card).
Frequency of Occurrence	How often this use case is triggered.
Miscellaneous	Open issues, unresolved questions.

Fully-Dressed Use Case — Process Sale:

Field	Content
Use Case Name	UC1: Process Sale
Scope	NextGen POS Application

Level	User-goal
Primary Actor	Cashier
Stakeholders & Interests	Cashier: fast, error-free entry. Customer: quick service, correct price. Manager: accurate sales records. Tax Authority: tax correctly calculated.
Preconditions	Cashier is identified and authenticated. POS terminal is operational.
Success Guarantee	Sale is recorded. Inventory updated. Payment processed. Receipt printed. Tax correctly calculated.
Main Success Scenario	1. Customer arrives at POS checkout with items. 2. Cashier starts new sale. 3. Cashier enters item identifier. 4. System records sale line item, displays description, price, running total. 5. Steps 3-4 repeated until done. 6. System displays total with taxes. 7. Cashier tells Customer the total. 8. Customer pays. System handles payment. 9. System logs completed sale, updates inventory. 10. System prints receipt. Customer leaves with items.
Extensions	3a. Invalid item ID: System signals error and rejects entry. 8a. Customer pays by cash: System calculates change due. 8b. Customer pays by card: System requests card authorization. 8c. Authorization fails: Cashier notifies customer of failure.
Special Requirements	Response time < 2 seconds. Touch-screen or keyboard input. Must support multiple currencies.
Tech Variations	3a. Item may be scanned by barcode or entered manually. 8a. Payment via cash, credit card, or mobile payment.
Frequency	Hundreds of times per day.

Q9. Describe the significance of Use Case Elaboration. Differentiate between Brief, Casual, and Fully-Dressed Use Case formats. [4+4] (2081 Chaitra)

Answer:

Significance of Use Case Elaboration

- Identifies all stakeholders and their interests — ensures no requirement is missed.
- Captures exception flows — edge cases that the system must handle.
- Provides basis for system design — main success scenario drives SSDs and interaction diagrams.
- Serves as acceptance test criteria — success guarantee = pass criteria.
- Communicates requirements clearly — bridges gap between clients and developers.
- Guides test case creation — each extension = a test case to verify.

Brief vs. Casual vs. Fully-Dressed Use Case Formats:

Format	Description	Length	When to Use
Brief	One-paragraph summary of main success scenario only. No alternatives, no details.	1-2 sentences to 1 paragraph	Early requirements elicitation, first pass analysis, low-risk use cases.
Casual	Informal multi-paragraph format. Covers main scenario and main alternatives in a narrative style. No rigid sections.	2-4 paragraphs	Middle ground — moderate detail needed without full formality.
Fully-Dressed	All sections completed formally: preconditions, success guarantee, all extensions, special requirements, tech variations, stakeholders, etc.	1-5+ pages	Important/complex use cases driving architecture. Any high-risk or high-value use case.

Recommended approach: Start all use cases as BRIEF. Elevate ~10% of the most important use cases to FULLY-DRESSED for the architecture design phase. Use CASUAL for intermediate priority use cases.

SECTION 5: Determining Visibility Between Objects

Exam Ref: 2082 Q8 [5], 2081 Chaitra Q9 [5], 2081 Ashwin Q8 [5], 2080 Chaitra Q8 [5], 2079 Ashwin Q8 [5], 2079 Chaitra Q8 [5], 2078 Chaitra Q8 [5], 2078 Baishakh Q8 [5], 2077 Chaitra Q8 [5]

Q10. What is visibility in OOD? Explain the four types of visibility with examples. [5] (2082 Kartik, 2081 Chaitra, 2079 Ashwin, 2078 Chaitra — VERY REPEATED)

Answer:

Visibility — Definition

Visibility refers to the ability of one object (sender) to see or have a reference to another object (receiver) so that it can send it a message. Before A can send a message to B, A must be 'visible' to B — A must have some way to reference B. Visibility is a prerequisite for sending messages in an interaction diagram.

Why Visibility Matters in Design:

- When drawing Collaboration/Sequence Diagrams, every message from A to B requires A to have visibility of B.
- Each type of visibility corresponds to different implementation choices in code.
- Determines how tightly objects are coupled.

Four Types of Visibility:

Type	Definition	How Achieved	Example	Coupling
1. Attribute Visibility	B is an attribute (field) of A. A permanently holds a reference to B.	B declared as instance variable in A.	Register has attribute currentSale:Sale. Register can always message Sale.	Highest — permanent reference
2. Parameter Visibility	B is passed as a parameter to a method in A. A can see B only while that method executes.	B passed as argument to A's method.	Sale.makeLineItem(spec:ProductDescription, qty). Sale can message ProductDescription during this method.	Moderate — temporary
3. Local Visibility	B is a locally created object inside a method of A. B exists only for that method.	B created or obtained as a local variable within A's method.	In enterItem(): spec = catalog.getSpecification(itemID). spec has local visibility.	Low — very temporary
4. Global Visibility	B is globally accessible to A. Rarely used in OO design.	B is a singleton, global variable, or class variable.	System-wide configuration or Logger accessible globally.	Lowest formality — avoid overuse

Visibility Example — POS enterItem() Operation:

Object Pair	Visibility Type	Justification
Register → Sale	Attribute Visibility	Register has currentSale:Sale as a field. Register permanently holds Sale reference.
Sale → ProductDescription (spec)	Parameter Visibility	spec is passed into Sale.makeLineItem(spec, qty) as a parameter.
Sale → ProductCatalog (catalog)	Attribute or Parameter	Catalog may be an attribute of Register (passed down) or Sale holds it directly.
Sale → SaleLineItem (sli)	Local Visibility	sli is created locally within Sale.makeLineItem() method.

Q11. Explain the concept of visibility in object-oriented design. How do you determine visibility in a Collaboration Diagram? [5] (2080 Chaitra, 2079 Chaitra, 2078 Baishakh)

Answer:

Determining Visibility in a Collaboration Diagram

Every link (line) between two objects in a Collaboration Diagram implies that the sender object has visibility of the receiver. When designing the collaboration diagram, for each message sent from A to B, you must decide HOW A will reference B — which type of visibility applies.

Decision Process:

Step 1: For each message from A to B in the collaboration diagram, ask: 'How does A get a reference to B?'
Step 2: If A always needs B (e.g., Register always needs Sale) → Attribute Visibility (make B a field of A).
Step 3: If B is only needed for one operation and comes from a parameter → Parameter Visibility.
Step 4: If B is created temporarily within a method → Local Visibility (local variable).
Step 5: Global visibility only if B is a system-wide singleton (e.g., logger).

Visibility → Implementation Mapping:

Visibility Type	Java/C++ Implementation
Attribute Visibility	private Sale currentSale; // field in Register class
Parameter Visibility	public void makeLineItem(ProductDescription spec, int qty) { ... }
Local Visibility	ProductDescription spec = catalog.getSpecification(itemID); // local variable
Global Visibility	Singleton.getInstance() or static field

SECTION 6: Class Diagram — Design Class Diagram (DCD)

Exam Ref: 2082 Q9 [10], 2081 Chaitra Q10 [10], 2081 Ashwin Q9 [10], 2080 Chaitra Q9 [10], 2080 Ashwin Q9 [10], 2079 Ashwin Q9 [10], 2078 Chaitra Q9 [10], 2078 Baishakh Q9 [10], 2077 Chaitra Q9 [10]

Q12. What is a Design Class Diagram (DCD)? How is it different from a Domain Model? What information does a DCD contain? [4+3+3] (2080 Chaitra)

Answer:

Design Class Diagram (DCD) — Definition

A Design Class Diagram (DCD) is a UML class diagram that shows the software classes (design classes), their attributes (with types and visibility), methods (with signatures), and their relationships — as they will be implemented in code. It is the primary artifact of OOD and directly guides implementation.

DCD vs. Domain Model — Key Differences:

Aspect	Domain Model (OOA)	Design Class Diagram (OOD)
Phase	Analysis (OOA)	Design (OOD)
Purpose	Conceptual map of real-world domain	Blueprint for software implementation
Classes shown	Conceptual classes (real-world things)	Software/Design classes (implementation objects)
Methods	NONE — no operations shown	ALL methods with full signatures
Attribute types	Optional / informal	Must have exact types (String, int, Date, etc.)
Visibility	NOT shown	SHOWN: + (public), - (private), # (protected)
Relationships	Conceptual associations, general	Navigability, multiplicity, dependency shown precisely
Pure Fabrications	Not present	Included (e.g., PersistenceService, TaxCalculator)
Implementation constructs	None (interfaces, generics not relevant)	Interfaces, abstract classes, generics shown

Information in a Design Class Diagram:

- Class name and stereotype (e.g., <<interface>>, <<abstract>>, <<controller>>).
- Attributes with type and visibility: - balance: Money, + name: String.
- Methods with full signatures: + getTotal(): Money, + enterItem(id: ItemID, qty: int): void.
- Associations with navigability arrows (one-way or bi-directional).
- Multiplicity (1, 0..1, 1..*, 0..*) on associations.
- Dependency relationships (dashed arrows) for uses/depends relationships.
- Inheritance (generalization) with hollow triangle arrows.
- Realization/Implementation (dashed + hollow triangle for interface implementation).

Q13. Draw the Design Class Diagram (DCD) for the POS System covering Process Sale use case. Apply all GRASP patterns in the design. [10] (2082 Kartik, 2081 Chaitra, 2078 Baishakh)

Answer:

Design Class Diagram — POS System (Process Sale)

The following describes the complete DCD for the POS Process Sale use case. Each class is described with attributes, methods, visibility, and relationships.

Class	Attributes	Methods	Relationships
Register	- currentSale: Sale	+ makeNewSale(): void + enterItem(id:ItemID, qty:int): void + endSale(): Money + makePayment(cash:Money): void	1 -- 1 Sale(attribute visibility) 1 -- 1 ProductCatalog 1 -- 1 Store
Sale	- date: Date - time: Time - isComplete: boolean - lineItems: List<SaleLineItem>	+ makeLineItem(spec:ProductDescription, qty:int): void + getTotal(): Money + becomeComplete(): void + makePayment(cash:Money): void + isComplete(): boolean	1 *-- 1..* SaleLineItem (composition) 1 -- 0..1 Payment 1 -- 1 Store
SaleLineItem	- quantity: int	+ getSubtotal(): Money	0..* --> 1 ProductDescription
ProductCatalog	- specifications: Map<ItemID, ProductDescription>	+ getSpecification(id:ItemID): ProductDescription + addSpec(spec:ProductDescription): void	1 *-- 0..* ProductDescription
ProductDescription	- description: String - price: Money - itemID: ItemID	+ getPrice(): Money + getDescription(): String + getItemID(): ItemID	Described by: ProductCatalog
Payment	- amount: Money	+ Payment(cashTendered:Money) + getAmount(): Money	1 -- 1 Sale
Store	- name: String - address: String	+ getProductCatalog(): ProductCatalog	1 *-- 1..* Register
CashPayment	- cashTendered: Money - changeGiven: Money	+ getChangeGiven(): Money + calculateChange(total:Money): Money	extends Payment (Generalization)
CardPayment	- cardNumber: String - authCode: String	+ authorize(): boolean	extends Payment (Generalization)

Key Design Decisions Applied:

- Information Expert: Sale.getTotal() — Sale has all SaleLineItems. SaleLineItem.getSubtotal() — SLI has qty and price.
- Creator: Register creates Sale. Sale creates SaleLineItem. Sale creates Payment.
- Controller: Register is Facade Controller for all 4 system operations.
- Low Coupling: Payment created by Sale (not Register) to avoid extra coupling.
- High Cohesion: Register only routes operations, domain logic stays in Sale.
- Polymorphism: Payment is abstract; CashPayment and CardPayment extend it.
- Protected Variations: Payment interface protects system from new payment type additions.

Q14. Draw and explain the Design Class Diagram for the following scenario: Library Management System (add book, issue book, return book, fine calculation). [10] (2081 Ashwin Back)

Answer:

Design Class Diagram — Library Management System

Class	Attributes	Key Methods	Key Relationships

Library (Facade Controller)	- name: String - address: String	+ addBook(isbn:String, qty:int): void + issueBook(isbn:String, memberID:String): Loan + returnBook(loanID:String): Fine + getMember(id:String): Member	1 *-- 1..* BookCopy 1 *-- 0..* Member 1 -- 1 Catalog
Book	- isbn: String - title: String - author: String - publisher: String - year: int	+ getISBN(): String + getTitle(): String + isAvailable(): boolean	1 *-- 1..* BookCopy (composition) 1 -- 1 BookDescription
BookCopy	- copyID: String - condition: String - isIssued: boolean	+ markIssued(): void + markReturned(): void + isAvailable(): boolean	0..* --> 1 Book
Member	- memberID: String - name: String - email: String - membershipType: String - maxBooks: int	+ getActiveLoans(): List<Loan> + canBorrow(): boolean	1 -- 0..* Loan
Loan	- loanID: String - issueDate: Date - dueDate: Date - returnDate: Date - status: String	+ isOverdue(): Boolean + getDaysOverdue(): int + markReturned(date:Date): void	1 --> 1 BookCopy 1 --> 1 Member 0..1 -- 0..1 Fine
Fine	- fineID: String - amount: Money - paidDate: Date - isPaid: boolean	+ calculateAmount(daysOverdue:int, rate:Money): Money + markPaid(): void	1 --> 1 Loan (Information Expert)
Catalog	- books: Map<String, Book>	+ searchByISBN(isbn:String): Book + searchByTitle(title:String): List<Book> + addBook(book:Book): void	1 *-- 0..* Book

SECTION 7: Additional Design Topics — Sequence Diagrams, Patterns & Case Studies

Exam Ref: 2080 Ashwin Q7, Q8, 2079 Jestha Q6, 2078 Poush Q6, 2077 Chaitra Q10, 2076, 2075, 2074

Q15. Draw a Sequence Diagram for the 'Return Book' use case of a Library Management System. [8] (2080 Ashwin Back)

Answer:

Sequence Diagram — Return Book Use Case

Participants: :Librarian (Actor), :Library (Controller), :Loan, :BookCopy, :FineCalculator (Pure Fabrication), :Fine

Step	Message	Sender → Receiver	Notes
1	returnBook(loanID)	Librarian → :Library	Librarian scans or enters loan ID
2	loan = findLoan(loanID)	:Library → :LoanRepository	Find the loan record
3	isOverdue = loan.isOverdue()	:Library → :Loan	Check if book returned late
4 [if overdue]	days = loan.getDaysOverdue()	:Library → :Loan	Get number of overdue days
4.1 [if overdue]	fine = calculateFine(days)	:Library → :FineCalculator	Pure Fabrication — dedicated fine service
4.2 [if overdue]	<<create>>(loan, amount)	:Library → :Fine	Creator: Library records Fine

5	loan.markReturned(today)	:Library → :Loan	Update loan status to returned
6	copy.markReturned()	:Loan → :BookCopy	Mark book copy as available
7	return receipt(loan, fine)	:Library → :Librarian	Return results to UI layer

Q16. What is the Singleton pattern? Where and how is it used in OOD? Explain with an example. [5] (2079 Jestha, 2078 Poush)

Answer:

Singleton Pattern — Definition

The Singleton Pattern is a creational design pattern that ensures a class has only ONE instance and provides a global point of access to that single instance. It is one of the GoF (Gang of Four) Design Patterns.

When to Use Singleton:

- When exactly one object is needed to coordinate actions across the system.
- When a shared resource must be managed by a single object.
- Common uses: Logger, Configuration Manager, Database Connection Pool, Print Spooler, Cache.

Singleton — Structure:

Element	Description
Private static instance	static instance: Singleton — the single instance, stored as class variable.
Private constructor	Prevents external instantiation — new Singleton() is not allowed.
Public static getInstance()	Returns the single instance. Creates it on first call (lazy initialization).

Java Code Structure of Singleton:

```
public class ProductCatalog {
        private static ProductCatalog instance = null;
        private ProductCatalog() { /* private constructor */ }
        public static ProductCatalog getInstance() {
                if (instance == null) {
                        instance = new ProductCatalog();
                }
                return instance;
        }
        public ProductDescription getSpecification(ItemID id) { ... }
}
```

Singleton in POS System:

- ProductCatalog is a Singleton — the entire POS system shares one catalog of products.
- Register retrieves it via: ProductCatalog catalog = ProductCatalog.getInstance();
- Avoids creating multiple catalog objects — saves memory and ensures data consistency.

Pros and Cons:

Advantages	Disadvantages
Controlled access to the single instance.	Can become a 'God Object' with too many responsibilities.
Saves memory — only one object created.	Hard to unit test (global state is hard to mock/replace).
Global access without global variables.	Violates Single Responsibility Principle if overused.
Lazy initialization possible.	Not thread-safe without synchronization in multithreaded environments.

Q17. Differentiate between Aggregation and Composition in UML Class Diagrams. Give an example of each in the context of a Library System. [5] (2077 Chaitra Q10, 2078 Poush)

Answer:

Aspect	Aggregation (Weak 'Has-A')	Composition (Strong 'Has-A')
Symbol	Hollow diamond (<>) at whole end	Filled/solid diamond (*) at whole end
Relationship strength	Weak — part CAN EXIST independently of whole	Strong — part CANNOT EXIST without whole
Lifecycle dependency	Independent lifecycle — part survives if whole is destroyed	Dependent lifecycle — destroying whole destroys parts
Ownership	Shared ownership possible	Exclusive ownership — only one whole owns the part
Library Example	Library <>—— Book (Book can exist outside library)	Book *—— BookCopy (BookCopy only makes sense within a Book)
Second Example	Course <>—— Student (Student exists without the course)	Order *—— OrderLine (OrderLine meaningless without Order)
Third Example	Team <>—— Player (Player exists after team dissolves)	House *—— Room (Room is destroyed if House is demolished)
UML Direction	Arrow at whole side: Library <>—— Book	Arrow at whole side: Book *—— BookCopy

---

Q18. Explain the four types of relationships in a UML Class Diagram: Association, Aggregation, Composition, and Generalization. [8] (2080 Ashwin, 2079 Ashwin)

Answer:

Relationship	UML Symbol	Meaning	Library Example	Code Implementation
Association	Solid line A — B (add arrow for navigability)	A structural link meaning 'A and B are related'. Can be uni or bidirectional.	Member — Loan (Member is associated with their loans)	Member has List<Loan> attribute (if navigable from Member to Loan)
Aggregation	Solid line + hollow diamond (<>) at whole end	Weak 'whole-part'. Part can exist independently. 'A has B'.	Library <>— Book (Books can exist without Library)	Library class has List<Book> field. Book can be referenced by other objects too.
Composition	Solid line + filled diamond (*) at whole end	Strong 'whole-part'. Part is owned exclusively and cannot exist without whole.	Book *— BookCopy (Copies destroyed when book removed)	BookCopy created inside Book constructor. BookCopy has no life outside Book.
Generalization	Solid line + hollow triangle (—□) pointing to parent	Inheritance 'is-a'. Child inherits all attributes and methods of parent.	CashPayment —□ Payment (CashPayment IS-A Payment)	class CashPayment extends Payment { ... }
Realization	Dashed line + hollow triangle (--□) pointing to interface	Class implements an interface. Class promises to provide the interface's methods.	CashPayment --□ IPayable (CashPayment implements IPayable)	class CashPayment implements IPayable { public void pay() {...} }
Dependency	Dashed arrow A --> B	Weaker than association. A uses B temporarily (parameter or local).	Sale --> ProductDescription (Sale uses it temporarily)	ProductDescription passed as method parameter to Sale.makeLineItem()

SECTION 8: Case Study Application — Complete OOD Design

Exam Ref: 2082 Q9 [10], 2081 Ashwin Q9 [10], 2080 Chaitra Q9, 2079 Ashwin Q9 [10], 2078 Baishakh Q9

Q19. For the Online Shopping/E-Commerce System: (a) Draw a Use Case Diagram. (b) Draw a System Sequence Diagram for 'Place Order'. (c) Apply GRASP patterns. (d) Draw a Design Class Diagram. [10] (2081 Ashwin Back)

Answer:

(a) Actors and Use Cases — Online Shopping System:

Actor	Use Cases
Customer (Primary)	Register/Login, Browse Products, Search Products, Add to Cart, Place Order (<<include>> Payment), Track Order, Cancel Order, Write Review
Admin	Login, Manage Products, Manage Orders, Manage Users, Generate Reports
Payment Gateway (External)	Process Payment (<<include>> in Place Order)
Delivery System (External)	Update Delivery Status (<<extend>> Track Order)

(b) SSD — Place Order:

Step	Event	Response
1	viewCart(): cartItems, total	Customer → System → cart displayed
2	placeOrder(cartID, addressID)	Customer → System → orderConfirmation(orderID)
3	selectPayment(orderID, paymentType)	Customer → System → paymentForm
4	processPayment(orderID, paymentDetails)	Customer → System → paymentResult
5 [if success]	confirmOrder(orderID)	System → Customer: orderConfirmed, estimatedDelivery
6	receiveNotification(orderID)	System → Customer: email/SMS sent

(c) GRASP Applied — Place Order:

System Operation	GRASP Pattern	Class Assigned	Reason
placeOrder()	Controller (Facade)	OrderController	Handles all order-related system events. Use Case Controller.
Creates Order	Creator	Cart creates Order	Cart records items that initialize the Order.
calculateTotal()	Information Expert	Order	Order contains OrderItems with prices. Expert has the data.
processPayment()	Pure Fabrication	PaymentService	Domain objects shouldn't know payment gateway details.
sendNotification()	Pure Fabrication	NotificationService	Domain classes shouldn't handle email/SMS directly.
PaymentType varies	Polymorphism + Protected Variations	Payment (abstract)	CashPayment/CardPayment/WalletPayment extend Payment.

(d) Design Class Diagram — E-Commerce System:

Class	Key Attributes	Key Methods	Relationships
Customer	- customerID: String - name: String - email: String	+ login(): Boolean + viewCart(): Cart + placeOrder(): Order	1 -- 0..* Order 1 -- 0..1 Cart

Cart	- cartID: String - createdAt: Date	+ addItem(prod, qty): void + removeItem(itemID): void + getTotal(): Money + checkout(): Order	1 *-- 0..* CartItem 1 -- 1 Customer
Order	- orderID: String - orderDate: Date - status: OrderStatus - total: Money	+ calculateTotal(): Money + confirmOrder(): void + cancelOrder(): void + getStatus(): OrderStatus	1 *-- 1..* OrderItem 1 --> 1 Payment 1 --> 1 Address
OrderItem	- quantity: int - priceAtPurchase: Money	+ getSubtotal(): Money	0..* --> 1 Product
Product	- productID: String - name: String - price: Money - stock: int	+ isAvailable(qty:int): Boolean + reduceStock(qty:int): void	1 *-- 0..* CartItem
Payment (abstract)	- paymentID: String - amount: Money - status: String	+ pay(): boolean (abstract) + refund(): boolean	0..1 -- 1 Order
CardPayment	- cardNumber: String - authCode: String	+ pay(): Boolean + authorize(): boolean	extends Payment
PaymentService	<<Pure Fabrication>> - gatewayURL: String	+ processPayment(payment:Payment): Boolean + refundPayment(payment:Payment): boolean	depends on Payment
NotificationService	<<Pure Fabrication>>	+ sendEmail(to:String, msg:String): void + sendSMS(phone:String, msg:String): void	depends on Order

QUICK REVISION: Chapter 3 — Key Points Summary

Topic	Key Points to Remember	Expected Marks
Analysis to Design Transition	OOA→OOD: Domain Model → DCD. Use Cases → Interaction Diagrams. System Operations → Controller methods. GRASP applied.	4–6
GRASP — Information Expert	Assign to class with the most data. Sale.getTotal() (Sale has lineItems). SaleLineItem.getSubtotal() (SLI has qty, price).	1–2 marks per pattern
GRASP — Creator	CAREH: Contains, Aggregates, Records, has Exclusively-initializing data, Highly uses. Sale creates SaleLineItem.	1–2
GRASP — Controller	Facade (whole system: Register) OR Use Case Session (ProcessSaleHandler). NOT domain class, NOT UI.	1–2
GRASP — Low Coupling	Minimize inter-class dependencies. Payment created by Sale, not Register.	1–2
GRASP — High Cohesion	Each class does one focused thing. Register doesn't do all the logic.	1–2
GRASP — Polymorphism	Behavior varies by type → use abstract class/interface + overriding. Payment → CashPayment, CardPayment.	1–2
GRASP — Pure Fabrication	Invented non-domain service class. PersistenceService, TaxCalculator. High cohesion, low coupling.	1–2
GRASP — Indirection	Intermediate object mediates. TaxCalculator between Sale and TaxSystem.	1–2
GRASP — Protected Variations	Stable interface around variation points. Payment interface protects from new payment types.	1–2
Collaboration Diagram	Objects as boxes. Links as lines. Messages numbered (1, 1.1, 1.2, 1.1.1). Shows structural connections.	6–10
Seq vs Collab Diagram	Same info, different layout. Sequence = time order (top-down). Collaboration = structural (numbered).	3–5
Visibility Types	4 types: Attribute (permanent field), Parameter (method arg), Local (local variable), Global (singleton).	5
Design Class Diagram	DCD has: classes + attributes (with types) + methods (with signatures) + visibility (+/-/#) + associations + navigability.	8–10
DCD vs Domain Model	Domain: no methods, no types, no visibility. DCD: all of these. DCD is implementation-ready blueprint.	3–5
Fully-Dressed Use Case	Sections: Name, Scope, Level, Primary Actor, Stakeholders, Preconditions, Success Guarantee, Main Scenario, Extensions, Special Reqs.	5–8
Singleton Pattern	One instance only. Private constructor. Static getInstance(). Used for: Catalog, Logger, Config.	3–5
Aggregation vs Composition	Aggregation (hollow diamond): part exists independently. Composition (filled diamond): part cannot exist alone.	3–5